



CredShields

Smart Contract Audit

June 2, 2026 • CONFIDENTIAL

Description

This document details the process and result of the Smart Contract audit performed by CredShields Technologies PTE. LTD. on behalf of Avenia between March 24, 2026, and March 27, 2026. A retest was performed on May 28, 2026.

Author

Shashank (Co-founder, CredShields) shashank@CredShields.com

Reviewers

Aditya Dixit (Research Team Lead), Shreyas Koli (Auditor), Sanket Salavi (Auditor), Prasad Kuri (Auditor), Neel Shah (Auditor)

Prepared for

Avenia



Table of Contents

Table of Contents	2
1. Executive Summary -----	4
State of Security	5
2. The Methodology -----	6
2.1 Preparation Phase	6
2.1.1 Scope	6
2.1.2 Documentation	6
2.1.3 Audit Goals	7
2.2 Retesting Phase	7
2.3 Vulnerability classification and severity	7
2.4 CredShields staff	9
3. Findings Summary -----	10
3.1 Findings Overview	10
3.1.1 Vulnerability Summary	10
5. Bug Reports -----	12
Bug ID #M001 [Fixed]	12
FLAG_ALLOW_REVERT Ineffective for Direct External Calls	12
Bug ID #M002 [Fixed]	14
V3_SWAP_EXACT_OUT Does Not Update CachedValue	14
Bug ID #M003 [Fixed]	16
Unchecked ERC20 Return Values via Raw .call()	16
Bug ID #L001 [Fixed]	18
1inch Fill Commands Do Not Update CachedValue	18
Bug ID #L002 [Fixed]	20
Floating and Outdated Pragma	20
Bug ID #L003 [Fixed]	21
Missing Input Length Validation in Assembly Decoding	21
Bug ID #L004 [Fixed]	22
Missing events in important functions	22
Bug ID #I001 [Fixed]	23
Redundant success Check in Direct Call Branches	23
Bug ID #I002 [Acknowledged]	25
Arbitrary External Call Capability via Operator	25
Bug ID #I003 [Acknowledged]	26
No Reentrancy Protection on External Calls	26
Bug ID #I004 [Fixed]	27
Missing zero address validations	27



Bug ID #I005 [Fixed]	28
Missing Storage Gap in UUPS Upgradeable Contract	28
Bug ID #I006 [Acknowledged]	29
Inconsistent Use of SafeERC20	29
6. The Disclosure -----	30



1. Executive Summary -----

Avenia engaged CredShields to perform a smart contract audit from March 24, 2026, to March 27, 2026. During this timeframe, 13 vulnerabilities were identified. **A retest was performed on May 28, 2026, and all the bugs have been addressed.**

During the audit, 0 vulnerabilities were found with a severity rating of either High or Critical. These vulnerabilities represent the greatest immediate risk to "Avenia" and should be prioritized for remediation, and fortunately, none were found.

The table below shows the in-scope assets and a breakdown of findings by severity per asset. Section 2.3 contains more information on how severity is calculated.

Assets in Scope	Critical	High	Medium	Low	info	Gas	Σ
MiniVM Smart Contracts	0	0	3	4	6	0	13

Table: Vulnerabilities Per Asset in Scope

The CredShields team conducted the security audit to focus on identifying vulnerabilities in Mini VM Contract's scope during the testing window while abiding by the policies set forth by Avenia's team.



State of Security

To maintain a robust security posture, it is essential to continuously review and improve upon current security processes. Utilizing CredShields' continuous audit feature allows both Avenia's internal security and development teams to not only identify specific vulnerabilities but also gain a deeper understanding of the current security threat landscape.

To ensure that vulnerabilities are not introduced when new features are added, or code is refactored, we recommend conducting regular security assessments. Additionally, by analyzing the root cause of resolved vulnerabilities, the internal teams at Avenia can implement both manual and automated procedures to eliminate entire classes of vulnerabilities in the future. By taking a proactive approach, Avenia can future-proof its security posture and protect its assets.



2. The Methodology -----

Avenia engaged CredShields to perform a Mini VM Contract audit. The following sections cover how the engagement was put together and executed.

2.1 Preparation Phase

The CredShields team meticulously reviewed all provided documents and comments in the smart contract code to gain a thorough understanding of the contract's features and functionalities. They meticulously examined all functions and created a mind map to systematically identify potential security vulnerabilities, prioritizing those that were more critical and business-sensitive for the refactored code. To confirm their findings, the team deployed a self-hosted version of the smart contract and performed verifications and validations during the audit phase.

A testing window from March 24, 2026, to March 27, 2026, was agreed upon during the preparation phase.

2.1.1 Scope

During the preparation phase, the following scope for the engagement was agreed upon:

IN SCOPE ASSETS
Audited Scope: https://etherscan.io/address/0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code

2.1.2 Documentation

Documentation was not required as the code was self-sufficient for understanding the project.



2.1.3 Audit Goals

CredShields employs a combination of in-house tools and thorough manual review processes to deliver comprehensive smart contract security audits. The majority of the audit involves manual inspection of the contract's source code, guided by OWASP's Smart Contract Security Weakness Enumeration (SCWE) framework and an extended, self-developed checklist built from industry best practices. The team focuses on deeply understanding the contract's core logic, designing targeted test cases, and assessing business logic for potential vulnerabilities across OWASP's identified weakness classes.

CredShields aligns its auditing methodology with the [OWASP Smart Contract Security](#) projects, including the Smart Contract Security Verification Standard (SCSVS), the Smart Contract Weakness Enumeration (SCWE), and the Smart Contract Secure Testing Guide (SCSTG). These frameworks, actively contributed to and co-developed by the CredShields team, aim to bring consistency, clarity, and depth to smart contract security assessments. By adhering to these OWASP standards, we ensure that each audit is performed against a transparent, community-driven, and technically robust baseline. This approach enables us to deliver structured, high-quality audits that address both common and complex smart contract vulnerabilities systematically.

2.2 Retesting Phase

Avenia is actively partnering with CredShields to validate the remediations implemented towards the discovered vulnerabilities.

2.3 Vulnerability classification and severity

CredShields follows OWASP's Risk Rating Methodology to determine the risk associated with discovered vulnerabilities. This approach considers two factors - Likelihood and Impact - which are evaluated with three possible values - **Low**, **Medium**, and **High**, based on factors such as Threat



agents, Vulnerability factors, and Technical and Business Impacts. The overall severity of the risk is calculated by combining the likelihood and impact estimates.

Overall Risk Severity				
Impact	HIGH	● Medium	● High	● Critical
	MEDIUM	● Low	● Medium	● High
	LOW	● None	● Low	● Medium
		LOW	MEDIUM	HIGH
Likelihood				

Overall, the categories can be defined as described below -

1. Informational

We prioritize technical excellence and pay attention to detail in our coding practices. Our guidelines, standards, and best practices help ensure software stability and reliability. Informational vulnerabilities are opportunities for improvement and do not pose a direct risk to the contract. Code maintainers should use their own judgment on whether to address them.

2. Low

Low-risk vulnerabilities are those that either have a small impact or can't be exploited repeatedly or those the client considers insignificant based on their specific business circumstances.

3. Medium

Medium-severity vulnerabilities are those caused by weak or flawed logic in the code and can lead to exfiltration or modification of private user information. These vulnerabilities



can harm the client's reputation under certain conditions and should be fixed within a specified timeframe.

4. High

High-severity vulnerabilities pose a significant risk to the Smart Contract and the organization. They can result in the loss of funds for some users, may or may not require specific conditions, and are more complex to exploit. These vulnerabilities can harm the client's reputation and should be fixed immediately.

5. Critical

Critical issues are directly exploitable bugs or security vulnerabilities that do not require specific conditions. They often result in the loss of funds and Ether from Smart Contracts or users and put sensitive user information at risk of compromise or modification. The client's reputation and financial stability will be severely impacted if these issues are not addressed immediately.

6. Gas

To address the risk and volatility of smart contracts and the use of gas as a method of payment, CredShields has introduced a "Gas" severity category. This category deals with optimizing code and refactoring to conserve gas.

2.4 CredShields staff

The following individual at CredShields managed this engagement and produced this report:

- Shashank, Co-founder CredShields shashank@CredShields.com

Please feel free to contact this individual with any questions or concerns you have about the engagement or this document.



3. Findings Summary -----

This chapter contains the results of the security assessment. Findings are sorted by their severity and grouped by asset and OWASP SCWE classification. Each asset section includes a summary highlighting the key risks and observations. The table in the executive summary presents the total number of identified security vulnerabilities per asset, categorized by risk severity based on the OWASP Smart Contract Security Weakness Enumeration framework.

3.1 Findings Overview

3.1.1 Vulnerability Summary

During the security assessment, 13 security vulnerabilities were identified in the asset.

Vulnerability Title	Severity	Vulnerability Type	Status
FLAG_ALLOW_REVERT Ineffective for Direct External Calls	Medium	Business Logic (SC03-LogicErrors)	Fixed
V3_SWAP_EXACT_OUT Does Not Update CachedValue	Medium	Business Logic (SC03-LogicErrors)	Fixed
Unchecked ERC20 Return Values via Raw .call()	Medium	Unchecked return value (SCWE-048)	Fixed
1inch Fill Commands Do Not Update CachedValue	Low	Business Logic (SC03-LogicErrors)	Fixed
Floating and Outdated Pragma	Low	Floating Pragma (SCWE-060)	Fixed
Missing Input Length Validation in Assembly Decoding	Low	Lack of Input Validation (SC04-Lack Of Input Validation)	Fixed
Missing events in important functions	Low	Missing Best Practices	Fixed
Redundant success Check in Direct Call Branches	Informational	Code Quality Issue	Fixed
Arbitrary External Call Capability via Operator	Informational	Design Risk	Acknowledged
No Reentrancy Protection on External Calls	Informational	Reentrancy Attacks (SCWE-046)	Acknowledged



Missing zero address validations	Informational	Missing Input Validation (SC04-Lack Of Input Validation)	Fixed
Missing Storage Gap in UUPS Upgradeable Contract	Informational	Upgradeability / Storage Layout Risk	Fixed
Inconsistent Use of SafeERC20	Informational	Code Quality	Acknowledged

Table: Findings and Remediations



5. Bug Reports -----

Bug ID #M001[Fixed]

FLAG_ALLOW_REVERT Ineffective for Direct External Calls

Vulnerability Type

Business Logic ([SC03-LogicErrors](#))

Severity

Medium

Description

The contract is designed to allow certain commands to fail without reverting the entire batch using the `FLAG_ALLOW_REVERT` flag. This relies on `dispatch()` returning a `(success, output)` value so that `execute()` can decide whether to continue.

However, some commands (`WRAP_NATIVE`, `UNWRAP_WNATIVE`, `V3_SWAP_EXACT_IN`, `V3_SWAP_EXACT_OUT`) use direct external calls instead of low-level `.call()`. If these calls revert, the revert propagates immediately and `dispatch()` does not return. As a result, the flag check in `execute()` is never reached.

This makes `FLAG_ALLOW_REVERT` ineffective for these commands and breaks the expected behavior of partial execution.

Affected Code

- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L91](#)

Impacts

Commands marked to allow failure may still revert the entire transaction. This breaks batch execution assumptions and can cause unexpected failures in automated workflows.

Remediation

Wrap these external calls in `try/catch` or use low-level `.call()` so that failures `return(success, output)` instead of reverting, ensuring consistent behavior across all commands.

Retest



All four affected commands (`WRAP_NATIVE`, `UNWRAP_WNATIVE`, `V3_SWAP_EXACT_IN`, `V3_SWAP_EXACT_OUT`) now use try/catch blocks. On failure, success is set to false and the revert reason is captured in output, allowing `execute()` to honour `FLAG_ALLOW_REVERT` correctly.



Bug ID #M002 [Fixed]

V3_SWAP_EXACT_OUT Does Not Update CachedValue

Vulnerability Type

Business Logic ([SC03-LogicErrors](#))

Severity

Medium

Description

The MiniVM contract uses a shared state variable `CachedValue` to pass values between commands within a batch. Many commands update this value after execution so that subsequent commands can reuse it via a sentinel value (`type(uint256).max - 1`).

However, in the `V3_SWAP_EXACT_OUT` command, the contract executes a swap using `UniswapRouter.exactOutput(params)` and obtains the actual input amount (`amountIn`) spent for the swap. Despite this, the contract does not update `CachedValue`.

As a result, `CachedValue` retains the value from a previous command, creating a state inconsistency. Any subsequent command relying on the sentinel value will use stale data instead of the actual amount involved in the swap.

This breaks the intended command chaining mechanism and leads to incorrect execution behavior.

Affected Code

- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L381](#)

Impacts

Subsequent commands that depend on `CachedValue` may operate on incorrect amounts, leading to failed transfers, incorrect accounting, or unexpected behavior in batch execution. This can disrupt automated workflows and break assumptions about command chaining.

Remediation

Update `CachedValue` after successful execution of the swap to reflect the actual amount used.

Retest



Inside the try `UniswapRouter.exactOutput(params)` returns (`uint256 amountIn`) block, `CachedValue = amountIn` is now assigned, ensuring subsequent commands receive the actual input amount spent by the swap.



Bug ID #M003 [Fixed]

Unchecked ERC20 Return Values via Raw `.call()`

Vulnerability Type

Unchecked return value ([SCWE-048](#))

Severity

Medium

Description

The contract performs ERC20 operations such as transfer, transferFrom, approve, mint, and burn using low-level `.call()` without validating the returned data. While `.call()` returns `success = true` if the call does not revert, it does not ensure that the ERC20 operation itself succeeded.

Some ERC20 tokens return false on failure instead of reverting. In such cases, the contract treats the operation as successful and proceeds to update `CachedValue`. This leads to incorrect state propagation, as subsequent commands may rely on values that were never actually transferred or processed.

Affected Code

- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L127](#)
- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L151](#)
- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L153](#)
- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L182](#)
- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L184](#)
- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L21](#)
- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L213](#)
- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L244](#)
- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L246](#)
- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L262](#)
- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L284](#)
- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F25#L284](#)

Impacts



Failed token operations may be treated as successful, causing incorrect `CachedValue` updates and leading to wrong execution in subsequent commands.

Remediation

Use SafeERC20 functions or validate the return data from `.call()`.

Retest

The issue is now Fixed.



Bug ID #L001[Fixed]

1inch Fill Commands Do Not Update CachedValue

Vulnerability Type

Business Logic ([SC03-LogicErrors](#))

Severity

Low

Description

The MiniVM contract uses a shared variable `CachedValue` to pass values between commands in a batch execution. Several commands update this value after execution so that subsequent commands can reuse it via a sentinel value `(type(uint256).max - 1)`.

However, the commands `ONE_INCH_FILL_ORDER` and `ONE_INCH_FILL_CONTRACT_ORDER` execute trades via low-level calls but do not update `CachedValue` after successful execution. As a result, even though tokens are swapped, the cached value remains unchanged from a previous command.

This creates inconsistency in command chaining, as subsequent commands relying on the cached value may operate on stale data instead of the actual output of the 1inch trade.

Affected Code

- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L445](#)
- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L479](#)

Impacts

Subsequent commands using the sentinel value may use outdated amounts, leading to incorrect transfers or unexpected behavior in batch execution.

Remediation

After successful execution, decode the returned data from the 1inch call and update `CachedValue` with the appropriate amount (e.g., the received token amount).

Retest

Both `ONE_INCH_FILL_ORDER` and `ONE_INCH_FILL_CONTRACT_ORDER` now check if `(success && output.length >= 96)`, decode the return tuple `(uint256 makingAmount, uint256, bytes32)`, and



assign `CachedValue = makingAmount`. Subsequent commands relying on the sentinel value will now receive the actual token amount from the 1inch trade.



Bug ID #L002 [Fixed]

Floating and Outdated Pragma

Vulnerability Type

Floating Pragma ([SCWE-060](#))

Severity

Low

Description

Locking the pragma helps ensure that the contracts do not accidentally get deployed using an older version of the Solidity compiler affected by vulnerabilities.

The contract allowed floating or unlocked pragma to be used, i.e., ^0.8.20. This allows the contracts to be compiled with all the solidity compiler versions above the limit specified. The following contracts were found to be affected -

Affected Code

- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L3](#)

Impacts

If the smart contract gets compiled and deployed with an older or too recent version of the solidity compiler, there's a chance that it may get compromised due to the bugs present in the older versions or unidentified exploits in the new versions.

Incompatibility issues may also arise if the contract code does not support features in other compiler versions, therefore, breaking the logic.

The likelihood of exploitation is low.

Remediation

Keep the compiler versions consistent in all the smart contract files. Do not allow floating pragmas anywhere. It is suggested to use the 0.8.34 pragma version

Reference: <https://scs.owasp.org/SCWE/SCSVS-CODE/SCWE-060/>

Retest

This issue is now Fixed.



Bug ID #L003 [Fixed]

Missing Input Length Validation in Assembly Decoding

Vulnerability Type

Lack of Input Validation (SC04-Lack Of Input Validation)

Severity

Low

Description

The contract decodes command inputs using inline assembly (`calldataload`) without checking that inputs has sufficient length. If the input is shorter than expected, `calldataload` reads out-of-bounds and returns zero-padded values.

This can lead to parameters such as `address(0)` or `amount = 0` being used silently. However, since `execute()` is restricted to `OPERATOR_ROLE`, only a trusted operator can supply such inputs.

Affected Code

- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L117](#)
- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L137](#)
- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L165](#)

Impacts

May lead to incorrect execution if malformed inputs are provided, but only under operator control.

Remediation

Add input length checks before decoding.

Retest

A custom `InputTooShort` error is defined and a minimum-length guard (`if (inputs.length < N) revert InputTooShort()`) is placed at the start of every command branch in `dispatch()`. All commands `MINT`, `BURN`, `BURN_BPS`, `TRANSFER`, `TRANSFER_BPS`, `APPROVE`, `PERMIT`, `EXECUTE_META_TRANSACTION`, `WRAP_NATIVE`, `UNWRAP_WNATIVE`, `V3_SWAP_EXACT_IN`, `V3_SWAP_EXACT_OUT`, `ONE_INCH_FILL_ORDER`, `ONE_INCH_FILL_CONTRACT_ORDER`, and `ETHEREUM_TO_GNOSIS_BRIDGE` are covered. Malformed short inputs will now revert explicitly rather than silently using zero-padded values.



Bug ID #L004 [Fixed]

Missing events in important functions

Vulnerability Type

Missing Best Practices

Severity

Low

Description

Events are inheritable members of contracts. When you call them, they cause the arguments to be stored in the transaction's log—a special data structure in the blockchain. These logs are associated with the address of the contract which can then be used by developers and auditors to keep track of the transactions.

The contract was found to be missing these events on certain critical functions which would make it difficult or impossible to track these transactions off-chain.

Affected Code

- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L59](#)
- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L63](#)
- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L67](#)

Impacts

Events are used to track the transactions off-chain and missing these events on critical functions makes it difficult to audit these logs if they're needed at a later stage.

Remediation

Consider emitting events for important functions to keep track of them.

Retest

Three events are now declared `WETHUpdated`, `UniswapRouterUpdated`, and `OmniBridgeUpdated` and each is emitted at the end of its corresponding setter (`setWETH`, `setUniswapRouter`, `setOmniBridge`). These state changes are now fully traceable on-chain.



Bug ID #I001[Fixed]

Redundant success Check in Direct Call Branches

Vulnerability Type

Code Quality Issue

Severity

Informational

Description

In the `dispatch()` function, the variable `success` is initialized to `true` and is not modified in certain command branches such as `WRAP_NATIVE`, `UNWRAP_WNATIVE`, and `V3_SWAP_EXACT_IN`. Despite this, these branches include a conditional check if `(success == true)` before updating `CachedValue`.

Since `success` is never reassigned in these branches, the condition is always `true` when execution reaches it. Additionally, if any of the direct external calls in these branches revert, the entire transaction reverts immediately and the conditional block is never executed.

As a result, the if `(success == true)` check is redundant and does not provide any functional value. It appears to be copied from other branches that use low-level `.call()` where `success` can be `false`, but in these direct-call branches, it is effectively dead code.

Affected Code

- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L108](#)

Impacts

This issue does not affect functionality or security but introduces misleading logic. It may confuse reviewers into assuming that failure handling exists in these branches, increasing code complexity and audit effort.

Remediation

Remove the redundant condition and directly assign the value.

Retest

he refactoring to try/catch for `WRAP_NATIVE`, `UNWRAP_WNATIVE`, `V3_SWAP_EXACT_IN`, and `V3_SWAP_EXACT_OUT` (as part of M001 remediation) naturally eliminates the redundant if `(success`



`== true`) guard. `CachedValue` is now updated directly inside the try block, and success is explicitly set to false only in the catch block.



Bug ID #I002 [Acknowledged]

Arbitrary External Call Capability via Operator

Vulnerability Type

Design Risk

Severity

Informational

Description

The MiniVM contract allows an address with `OPERATOR_ROLE` to execute arbitrary commands through the `execute()` function. Many of these commands internally use low-level `.call()` with user-controlled target addresses and calldata.

This gives the operator the ability to interact with any external contract and execute arbitrary functions using the contract's funds and approvals. There are no restrictions or validations on the target address or function being called.

This behavior appears intentional as part of the flexible execution design. However, it introduces a broad execution surface and increases reliance on the trustworthiness of the operator.

Affected Code

- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L77](#)

Impacts

There is no direct impact under a trusted operator assumption. However, if the operator is compromised or misuses privileges, they can execute arbitrary calls that may lead to fund loss or unintended interactions.

Remediation

Consider adding optional safeguards such as whitelisting trusted contracts or restricting callable functions. At minimum, clearly document the trust assumptions around the operator role.

Retest

Client comment: "The consequences of a compromised operator wallet are well known and for now we plan to address it by ensuring the adequate security and usage of the operator wallets (MPC, multisig, possibility to rotate from time to time)."



Bug ID #I003 [Acknowledged]

No Reentrancy Protection on External Calls

Vulnerability Type

Reentrancy Attacks ([SCWE-046](#))

Severity

Informational

Description

The MiniVM contract performs multiple external calls across different command branches, including interactions with ERC20 tokens, Uniswap router, 1inch, OmniBridge, and arbitrary contracts via low-level `.call()`. However, the contract does not implement any reentrancy protection such as a `nonReentrant` modifier.

Despite the presence of these external calls, the `execute()` function is restricted by `onlyRole(OPERATOR_ROLE)`, which prevents unauthorized entities from reentering the contract and executing further commands. Additionally, the contract does not maintain sensitive internal accounting that could be exploited through reentrancy.

As a result, while the contract technically has external call surfaces, there is no practical reentrancy attack vector under the current design and trust assumptions.

Affected Code

- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L77](#)

Impacts

There is no direct security impact in the current implementation. However, the absence of reentrancy protection may increase risk if the contract is upgraded in the future to include sensitive state changes or if role assumptions change.

Remediation

Consider adding reentrancy protection (e.g., `ReentrancyGuard`) as a defense-in-depth measure.

Retest

Client comment: "Some informational issues, such as reentrancy, we've kept as it is due to the operator role protection."



Bug ID #I004 [Fixed]

Missing zero address validations

Vulnerability Type

Missing Input Validation ([SC04-Lack Of Input Validation](#))

Severity

Informational

Description:

The contracts were found to be setting new addresses without proper validations for zero addresses.

Address type parameters should include a zero-address check otherwise contract functionality may become inaccessible or tokens burned forever.

Depending on the logic of the contract, this could prove fatal and the users or the contracts could lose their funds, or the ownership of the contract could be lost forever.

Affected Code

- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L43](#)
- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L59](#)
- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L63](#)
- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L67](#)

Impacts

If address type parameters do not include a zero-address check, contract functionality may become unavailable or tokens may be burned permanently.

Remediation

Add a zero address validation to all the functions where addresses are being set.

Retest

Zero-address checks have been added to all three setter functions ([setWETH](#), [setUniswapRouter](#), [setOmniBridge](#)) and to [initialize\(\)](#) for the defaultAdmin parameter. The omission of zero-address checks for [_wnative](#) and [_uniswapRouter](#) in [initialize\(\)](#) is intentional by design: these variables may legitimately be [address\(0\)](#) on chains where the respective contracts are not deployed. The setters, which are the primary post-deployment configuration path, are fully protected.



Bug ID #I005 [Fixed]

Missing Storage Gap in UUPS Upgradeable Contract

Vulnerability Type

Upgradeability / Storage Layout Risk

Severity

Informational

Description

The contract uses the UUPS upgradeable pattern but does not include a storage gap (`__gap`) after its state variables. Storage gaps are commonly used to reserve slots for future variables and reduce the risk of storage layout issues during upgrades.

Although this does not affect the current implementation, incorrect future upgrades (e.g., inserting variables in between existing ones) can lead to storage collisions and unintended state corruption.

Affected Code

- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L1](#)

Impacts

No immediate impact. Poorly implemented future upgrades may corrupt storage and break contract behavior.

Remediation

Add a storage gap to reserve space for future variables

Retest

A `uint256[50] private __gap` storage gap is now declared after the contract's state variables, reserving 50 storage slots for safe future upgrades without risk of storage layout collisions.



Bug ID #I006 [Acknowledged]

Inconsistent Use of SafeERC20

Vulnerability Type

Code Quality

Severity

Informational

Description

The contract declares using SafeERC20 for IERC20, but only a single location in V3_SWAP_EXACT_OUT – actually uses safeTransfer(). Every other ERC20 interaction across MINT, BURN, TRANSFER, APPROVE, and PERMIT commands uses raw .call() with abi.encodeWithSignature. This inconsistency suggests an incomplete migration to safe patterns and increases the probability of introducing return-value bugs.

Affected Code

- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L19](#)
- [0xc5604265f0f02b7dc697a3c8903bf046061f1f45#code#F1#L411](#)

Impacts

Inconsistent safety guarantees across commands. Code review difficulty increased due to mixed patterns.

Remediation

Adopt SafeERC20 methods uniformly across all ERC20 operations.

Retest

Client Comment : I006 was ack in our current context.



6. The Disclosure -----

The Reports provided by CredShields are not an endorsement or condemnation of any specific project or team and do not guarantee the security of any specific project. The contents of this report are not intended to be used to make decisions about buying or selling tokens, products, services, or any other assets and should not be interpreted as such.

Emerging technologies such as Smart Contracts and Solidity carry a high level of technical risk and uncertainty. CredShields does not provide any warranty or representation about the quality of code, the business model or the proprietors of any such business model, or the legal compliance of any business. The report is not intended to be used as investment advice and should not be relied upon as such.

CredShields Audit team is not responsible for any decisions or actions taken by any third party based on the report.



Your **Secure Future** Starts Here



At CredShields, we're more than just auditors. We're your strategic partner in ensuring a secure Web3 future. Our commitment to your success extends beyond the report, offering ongoing support and guidance to protect your digital assets



 SCS Advocates

